

MxR4DOT: A Residue MX Format in a RISC-V Coprocessor for Evaluating Hardware Cost Against Existing MX Formats **[TODO: working title]**

Anonymous Submission
Paper ID: **[TODO: SoftConf]**

Abstract—Four-bit Microscaling (MX) formats reach acceptable accuracy for many inference workloads, but some models and deployment targets demand more precision than a single four-bit block provides, without conceding the memory advantage that motivates four-bit weights in the first place. Several augmented MX variants address this at the format level; their hardware cost, however, is seldom measured against the alternatives they displace on common silicon.

This work introduces MXFP4R, an MX variant that represents activations as the sum of two MXFP4 blocks under independent block scales while keeping weights in plain MXFP4, and evaluates whether it merits a hardware implementation. Because only activations carry the second block, weight footprint remains that of MXFP4 while representational precision approaches that of MXFP8. The value of this trade depends on delivered throughput, which a format-level analysis cannot establish.

We implement MXFP4R alongside MXFP4, MXFP8, and M²XFP4 as format-specialised independent dot-product engines within a single CV-X-IF-compliant coprocessor (CV32E40X), characterising all four on one host core, one synthesis flow, and one technology node. Each engine accumulates in a scale-free frame whose width depends on the element format alone.

Synthesised at 45 nm, the specialised MXFP4 engine sustains $5.1 \times$ the throughput per unit area of MXFP8. The MXFP4R datapath requires three source operands and two block scales, exceeding a three-read-port register file, and is expressed as an instruction pair whose present throughput is **[DATA: X]** $\times$ that of MXFP8; its advantage therefore lies in halved weight memory at comparable accuracy rather than in compute.

Index Terms—Microscaling formats, MXFP8, RISC-V, CV-X-IF, dot product, low-precision arithmetic, ASIC synthesis.

I. INTRODUCTION

Microscaling (MX) formats, standardised by the Open Compute Project [1], associate a block of k elements with one 8-bit power-of-two exponent (E8M0) and encode each element in a narrow floating-point type. Four-bit MX has been shown adequate for a range of inference and even training workloads [2], yet its single shared power-of-two scale bounds the precision available within a block. Workloads sensitive to this bound have motivated augmented four-bit formats: M²XFP4 [3] allocates per-subgroup metadata to recover precision, and NVFP4 format replaces the conventional E8M0 block scale with a higher-precision FP8 block scale. These are proposed and evaluated as data formats; their realisation cost, measured against the formats they would replace on identical hardware, is largely absent from the literature.

This work examines a further variant that we propose, MXFP4R, in which an activation block is represented as the

sum of two MXFP4 blocks carrying independent E8M0 scales, while weight blocks remain plain MXFP4. The construction is asymmetric by design: weight footprint is that of MXFP4, whereas activation precision approaches that of MXFP8. For inference, where weights dominate memory and bandwidth, this is a favourable position *if* the arithmetic can be executed at competitive throughput.

That condition is the subject of the paper. A MXFP4R dot product forms two product terms under two independent scales and therefore needs three genuine 64-bit vector operands and two independent block scales at once, exceeding a three-read-port register file. The resulting throughput cannot be inferred from the format definition and must be measured on synthesised hardware. We do so by implementing MXFP4R together with MXFP4, MXFP8, and M²XFP4 as independent, format-specialised engines behind one coprocessor, and characterising all four on a common core, synthesis flow, and technology node.

A. Related work

Two RISC-V extensions for MX dot products precede this work. MXDOTP [4] adds an MXFP8 dot-product unit to a Snitch core and supplies block scales through Stream Semantic Registers. VMXDOTP [5] extends the RISC-V Vector specification to cover MXFP8 and MXFP4, resolving operand-port pressure by time-multiplexing vector register-file reads, and reports 250 GFLOP/s at 1632 GFLOP/(s W) for MXFP4 in 12 nm. This work targets a different point in the design space: a coprocessor behind the standardised CV-X-IF interface, hosted by a compact embedded core, covering formats these extensions do not. Neither prior extension supports M²XFP4 nor our proposed MXFP4R.

The accumulation datapath common to all three works derives from the anchored fixed-point scheme of Lutz *et al.* [6], which MXDOTP extended to the two independent block scales of an MX dot product. The present contribution is the derivation of this scheme for four distinct formats, each with its own frame geometry.

M²XFP4 was introduced as an algorithm–hardware co-design with a systolic-array accelerator [3]; the engine described here is a RISC-V coprocessor realisation that re-derives the per-subgroup top-1 element within the datapath.

B. Contributions

- 1) MXFP4R, a residue MX format, and its hardware evaluation: on a common 45 nm flow, its throughput relative to MXFP8 is $\frac{[DATA: X]}{(\$??)}$, locating its benefit in weight memory (§III) rather than compute.
- 2) A four-format comparison of MXFP4, MXFP8, M²XFP4, and MXFP4R on one host core and one synthesis flow, removing the process and system-scope differences that complicate cross-paper MX comparisons (§V).
- 3) A per-format scale-free accumulation frame, with anchor and width derived in closed form for each format and shown tight against exact rational arithmetic over the full E8M0 scale range (§IV-C).
- 4) A coprocessor conforming to the CV-X-IF interface, portable across compliant host cores (§II-B).

II. BACKGROUND

A. Microscaling formats

An MX block comprises k elements sharing a single E8M0 scale 2^{S-127} , $S \in [0, 254]$. The OCP specification fixes $k = 32$; the architectural granularity in this design is the 16-element half-block carried by a 64-bit dual-read operand pair. Elements are E2M1 for MXFP4 and E4M3 or E5M2 for MXFP8. For two blocks A, B with scales $s_a = 2^{S_a-127}$, $s_b = 2^{S_b-127}$ accumulating into an FP32 value, the scaled dot product is

$$acc' = s_a s_b \sum_i a_i b_i + acc. \quad (1)$$

Both scales are powers of two, so applying them is an exponent adjustment rather than a multiplication. Rewriting (1) to expose this,

$$acc' = s_a s_b \left(\sum_i a_i b_i + \frac{acc}{s_a s_b} \right), \quad (2)$$

places the products at a fixed fixed-point position and admits the accumulator by a shift of $E_{acc} - \text{bias} - (S_a + S_b - 254)$ bit positions, where E_{acc} is the accumulator's biased exponent. No multiplier is required on the datapath; the block scale reappears only in the final exponent. Equation (2) is the identity every engine realises.

B. Coprocessor interface

The host core, CV32E40X, is designed within OpenHW Group's CORE-V family to extend the ISA through an external coprocessor over CV-X-IF rather than by embedding custom functional units in its own pipeline, which keeps the coprocessor's cost isolated and measurable rather than conflated with changes to the host itself. The coprocessor attaches over CV-X-IF [7] and uses only the standardised issue, commit, and result channels, with no private path into the host pipeline; any XIF compliant core supporting three source reads and dual-read can host it. The host in this work, CV32E40X, does not implement CV-X-IF's optional dual-read, which was added to its register-file read path; the modification is independent of the coprocessor, which uses only the standard interface.

C. Motivation for MXFP4R

MXFP4 halves weight footprint versus MXFP8 but at a real cost in accuracy [3]; uniform MXFP8 recovers that accuracy but surrenders the memory-bandwidth advantage and widens the accumulation datapath. Asymmetric precision offers a middle path. On GPT-2/WikiText-2, naive MXFP4 collapses perplexity from 30.3 to 107.8; adding a second MXFP4 block to activations alone (MXFP4R, averaging 6.375 b/element) recovers 93% of that loss, against 36% for the mirrored weight-side allocation. This activation-side advantage is not universal, however: §III shows where it holds, where it inverts, and why.

III. MXFP4R ANALYSIS

Whether MXFP4R merits implementation depends on its post-training accuracy across model types. Table I reports the four implemented formats, NVFP4, and two single-sided ablations across nine models in three modalities.

Quantisation Protocol. Quantisation is post-training and calibration-free: weights use static per-block scaling ($k = 32$, OCP), activations use dynamic scaling from each tensor's peak magnitude, and all formats round to nearest-even. Language model evaluations average two seeds; vision and speech use a single seed (Table I, footnote).

Symmetric residue matches BF16 and MXFP8 accuracy everywhere, confirming that two independent MXFP4 blocks are an adequate substitute for one MXFP8 block. Restricting the residue to activations alone, however, is not universally optimal: it recovers most of the accuracy gap on seven of nine evaluated models, including GPT-2, all three larger language models, and the vision transformer, but the preference inverts on the deeper convolutional network and the speech model, where naive MXFP4 already collapses badly and weight-side residue instead dominates recovery. The inversion is not explained by architecture family or parameter volume: two closely related convolutional networks land on opposite sides despite near-identical weight-to-activation ratios. A per-layer error diagnostic offers real mechanistic insight (below), though it explains why an allocation fails more reliably than it predicts which allocation will win.

For the speech model, neither candidate explanation holds up: a sign-correlation test rules out error cancellation between weight and activation terms, and the error is not concentrated in the audio front end. The actual mechanism is subtler: activation-side residue collapses a large but symmetric, self-averaging quantisation error at one late, decoder-critical layer, but in doing so leaves a smaller, systematic per-channel weight bias as the dominant remaining error there instead. That structured bias proves more damaging to sequence decoding than the larger, unstructured noise it replaces. The result generalises beyond this format: reducing reconstruction error is not always sufficient when the reduction changes the character of the error, not just its size, at a decision-critical layer.

MXFP4R offers two single-sided configurations, placing the residue block on activations or on weights, and their

TABLE I

POST-TRAINING QUANTISATION ACCURACY. WIKITEXT-2 PERPLEXITY ($\downarrow$), IMAGENET-1K TOP-1 ($\uparrow$), LIBRISPEECH TEST-CLEAN WER ($\downarrow$). ALL LAYERS QUANTISED; BF16 IS THE UNQUANTISED REFERENCE. THE TWO RULES BELOW MXFP4R ARE SINGLE-SIDED AND SYMMETRIC ABLATIONS OF THE RESIDUE CONSTRUCTION.

Format	Perplexity $\downarrow$				Top-1 $\uparrow$		WER $\downarrow$
	GPT-2	Llama-2 7 B	Llama-3 8 B	Mistral 7 B	ViT-B $\dagger$	RN-50 $\dagger$	wav2vec2 $\dagger$
BF16	30.3	6.02	6.74	6.14	80.3	80.4	2.86
MXFP4	107.8	7.83	9.19	7.90	73.6	11.3	9.00
NVFP4	36.3	6.38	7.59	6.46	77.3	55.5	3.68
M ² XFP4	46.4	6.46	7.85	6.35	78.4	60.3	5.00
MXFP4R (residue on A)	35.4	6.65	7.75	6.49	78.1	27.2	30.5
<i>residue on W</i>	79.7	6.73	7.82	6.77	77.2	67.3	4.27
<i>residue on both</i>	30.3	6.04	6.79	6.15	80.3	79.3	2.77
MXFP8-E4M3	31.1	6.08	6.87	6.20	79.5	77.9	2.71

Underline: best among MXFP4 / NVFP4 / M²XFP4 / MXFP4R (residue on A) per column (BF16 and MXFP8-E4M3 are reference points, not candidates). MXFP4R (default) places residue on activations only (W/A = 4.25/8.50 b); “residue on W” is mirrored (8.50/4.25 b) and “residue on both” is symmetric (8.50/8.50 b). $\dagger$ vision/speech: single seed (42).

TABLE II

ZERO-SHOT TASK ACCURACY (%) ON DOWNSTREAM BENCHMARK EVALUATIONS FOR GPT-2 AND LLAMA-2-7B. BOLD INDICATES THE HIGHER-PERFORMING SINGLE-SIDED RESIDUE ALLOCATION.

Model	Task	BF16	MXFP4	M ² XFP4	MXFP4R (Act)	MXFP4R (W)
GPT-2	WinoGrande	51.6	48.5	49.8	52.6	51.7
	PIQA	62.3	58.2	60.1	61.4	58.3
	HellaSwag	31.3	29.0	30.6	29.9	29.8
	ARC-Easy	40.2	38.3	37.8	39.5	38.7
	BoolQ $\dagger$	50.2	59.6	57.6	57.6	56.7
Llama-2-7B	WinoGrande	69.1	66.5	67.6	68.8	68.1
	PIQA	78.4	76.9	77.8	77.9	77.5
	HellaSwag	76.1	70.6	74.5	74.1	74.0
	ARC-Easy	74.1	69.4	73.4	72.3	71.3
	BoolQ	79.4	72.8	77.3	77.1	76.6

$\dagger$ GPT-2 unquantised (BF16) accuracy on BoolQ is statistically indistinguishable from the 50% chance baseline; nominal gains under quantisation represent noise and are excluded from average gap recovery calculations.

strengths fall on different workloads. The activation-side variant achieves the lowest perplexity among all four-bit candidates on the compact decoder (GPT-2) and outperforms M²XFP4 on Llama-3-8B, though NVFP4 leads on the larger language models overall. The weight-side variant wins outright on the deeper convolutional network and outperforms M²XFP4 on the speech model. The two formats are therefore complementary rather than competing, motivating exposing both as independent coprocessor engines (§IV-D).

A. Zero-shot task validation

Five zero-shot reasoning benchmarks (Table II) corroborate the perplexity-level findings: activation-side residue outperforms weight-side residue on four of five tasks for both the compact decoder and the instruction-tuned model, recovering 78% and 67% of the quantisation gap respectively (the compact decoder’s near-chance BoolQ baseline excluded). M²XFP4 follows the same ordering under zero-shot evaluation as under perplexity, trailing activation-only residue on the compact decoder but competitive on the instruction-tuned model.

IV. MICROARCHITECTURE

A. Residue and metadata-augmented variants

MXFP4R adds a second activation block ar with its own scale $s_{ar} = 2^{S_{ar}-127}$, quantising the residual of the primary block against plain MXFP4 weights. The dot product carries two independent scales,

$$acc' = s_a s_w \sum_i a_i w_i + s_{ar} s_w \sum_i ar_i w_i + acc, \quad (3)$$

which factors on the primary scale as

$$acc' = s_a s_w \left(\sum_i a_i w_i + \frac{s_{ar}}{s_a} \sum_i ar_i w_i + \frac{acc}{s_a s_w} \right). \quad (4)$$

The ratio $s_{ar}/s_a = 2^{S_{ar}-S_a}$ is again a shift, by $\delta = S_a - S_{ar} \geq 0$; §IV-E establishes the non-negativity and its role in placement. The primary product term reuses the MXFP4 datapath unchanged.

M²XFP4 [3] retains E2M1 elements and adds per-instruction metadata. Within each 8-element subgroup, two bits extend the largest-magnitude activation element to E2M3, and two bits refine the weight subgroup scale to $(1 + \kappa/4) 2^S$, $\kappa \in \{0, 1, 2, 3\}$. The scaled sum of products becomes

$$acc' = s_a s_w \sum_g \left(1 + \frac{\kappa_g}{4} \right) \sum_{i \in g} a_i w_i + acc, \quad (5)$$

in which a_i denotes the top-1 element’s E2M3-corrected value for the subgroup’s single largest-magnitude activation and its plain E2M1 value otherwise, and the κ_g factor is a per-subgroup shift-and-add on the partial sum, not a multiplication.

B. Instruction interface

Instructions reuse the R4-type FMADD.S operand layout, so a third source needs no host register-file change.

`mx fused` is a single-instruction operation for MXFP4, MXFP8, and M²XFP4. MXFP4R instead needs three genuine 64-bit sources and two independent block scales at once, which exceeds a three-read-port register file, so it is issued as the

TABLE III

INSTRUCTION SET AND OPERAND MAP. R4-TYPE, CUSTOM-0 OPCODE 0x0B; FUNCT2 SELECTS THE MX FORMAT, FUNCT3 THE OPERATION. MXFP4R REQUIRES TWO NATIVE INSTRUCTIONS, SINCE IT CARRIES A THIRD 64 bit SOURCE AND A SECOND BLOCK SCALE (§IV-E).

Format	Mnemonic	f3/f2 [†]	Elms [‡]	rs1	rs2	rs3
MXFP4	mx fused	011/00	16	p_A	p_W	$\{x_W, x_A, acc\}$
MXFP8	mx fused	011/11	8	p_A	p_W	$\{x_W, x_A, acc\}$
M ² XFP4	mx fused	011/10	16	p_A	p_W	$\{md, x_W, x_A, acc\}$
MXFP4R	mxr4dot	000/01	16	p_A	p_W	p_{Ar}
	mx final	001/01	—	$\{x_A, x_{Ar}, x_W\}$	acc	—

[†]funct3/funct2. [‡]Elements per instruction. p_A, p_W, p_{Ar} : packed activation/weight/residue-activation vectors; x_A, x_{Ar}, x_W : E8M0 block scales; acc : FP32 accumulator; braces denote fields packed into one register word. p_A, p_W, p_{Ar} and mx fused's rs3 are 64 bit dual-read register pairs; mx final's rs1, rs2 are plain 32 bit reads. mxr4dot writes no register; mx final and mx fused write the FP32 result.

mxr4dot/mx final pair (§IV-E). For M²XFP4, operand order is significant: rs1 carries activations and rs2 weights.

C. Scale-free accumulation frame

Every engine realises (2) through a fixed-point frame in which the sum of products is resident and the accumulator, rather than the products, absorbs the block scale [4], [6]. The sum of products occupies a format-specific anchor position, where frame bit b carries weight $2^{b-\alpha}$ and the anchor α is chosen so that the smallest representable quantum aligns with bit 0; every product is then frame-resident without a shifter. The anchor thus tracks each format's smallest quantum: $\alpha = 2$ for MXFP4's E2M1 products, 32 for MXFP8, whose smallest E5M2 subnormal product is 2^{-32} ,¹ and 6 for M²XFP4, set by the $1/64$ quantum its $(1 + \kappa/4)$ refinement introduces; MXFP4R shares MXFP4's anchor. The FP32 accumulator enters by a shift of $E_{acc} - C_\alpha - (S_a + S_b - 254)$, where $C_\alpha = 150 - \alpha$ is a compile-time constant, and bits below the frame are retained in a REMAIN-bit extension with a sticky bit. The scale contributes only to the final exponent.

The consequence is that frame width is a function of the element format alone and is independent of the E8M0 scale range, so each engine is exact to round-to-nearest-even over every legal scale pair and every FP32 accumulator at a width set by its own format. Applying the scheme across four formats requires a distinct frame geometry for each; the constants are derived in closed form and listed in Table IV.

Three of the table's columns are linked as consistency checks: Frame = 26 + Σ_P (sign, 24-bit accumulator mantissa, guard bit, and the Σ_P field) and LZC = Frame + REMAIN (the leading-zero counter spans the full pre-normalisation word, not the frame alone); both hold in every row.

Two of the derived bounds are non-obvious. The REMAIN extension must guarantee that, under catastrophic cancellation with the sticky bit asserted, the surviving leading one cannot fall below the round-bit position; this bound depends only on the 24-bit accumulator mantissa, not on anchor or sum-of-products width, giving REMAIN = 25 identically for

TABLE IV

DERIVED SLIDING-FRAME CONSTANTS. FRAME WIDTH IS SET BY THE ELEMENT FORMAT ALONE; THE E8M0 SCALE RANGE DOES NOT APPEAR.

Engine	Anchor α	Σ_P^* (bits)	Frame (bits)	REMAIN (bits)	LZC (bits)
MXFP4	2	12	38	25	63
MXFP8	32	69	95	25	120
M ² XFP4	6	17	43	25	68
MXFP4R	2	12 [†]	38	36	74

*Unsigned sum-of-products field resident in the frame word. [†]Two product terms at independent scales (§IV-E); the wider REMAIN follows from cancellation between them rather than against the accumulator.

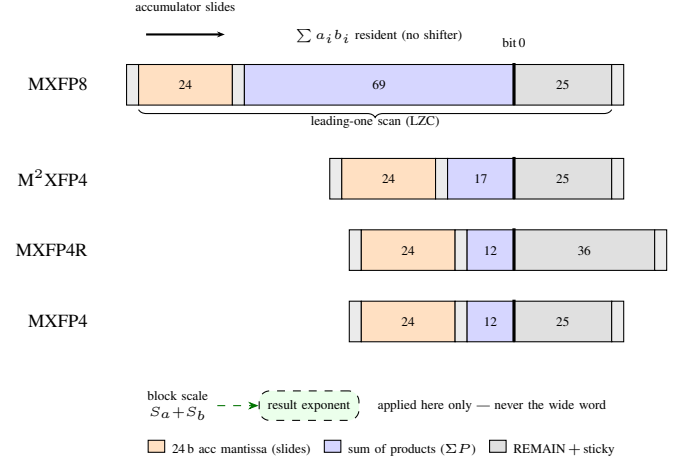


Fig. 1. Per-format scale-free accumulation frame, drawn to scale and aligned at bit 0. The sum of products is resident and exact at each format's anchor; only the FP32 accumulator slides (Table IV lists all widths).

MXFP4, MXFP8, and M²XFP4, with the two-product residue frame widening it (§IV-E). Consequently, no wider extension is required, avoiding unnecessary leading-zero detection width. Frame width follows a similar no-overflow argument bounding the accumulator against the resident sum of products, so no in-frame saturation logic is instantiated in any engine.

D. Format-specialised engines

Each format is realised as an independent engine instantiating the frame at its own constants. Width-parameterised blocks—leading-zero count, normalise-and-round, and the accumulator slide—are shared as modules elaborated at each format's width, so MXFP4 instantiates a 63-bit leading-zero count where MXFP8 instantiates a 120-bit one. The front-end decode, multiply array, and reduction tree are not shared, as these are format-specific: a single MXFP8 engine's decode covers both E4M3 and E5M2 elements, and unifying M²XFP4's widened top-1 lane (§IV-F) with MXFP4's narrower lanes into one common array would impose the wider format's cost on the narrower one. Preserving per-format module instances also allows area and timing to be attributed per format (§V).

Each fused engine is a five-stage pipeline—input capture, front-end output, frame word, leading-one result, and result register—with a single global stall that freezes all stages when

¹The reference implementation [4] anchors at 34; its per-element decode pads the E5M2 mantissa by one bit, shifting the same 95-bit frame up two positions, so the two are value-identical.

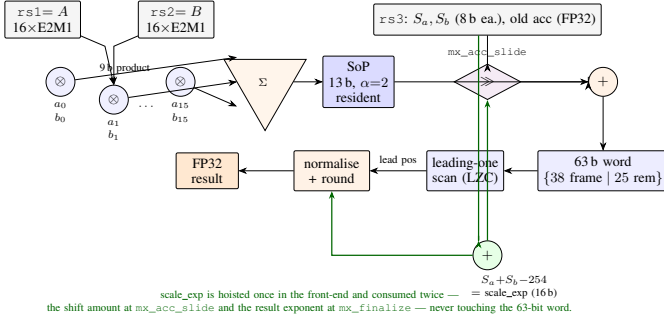


Fig. 2. MXFP4 fused-engine datapath, the exemplar of the four format-specialised engines. Sixteen lanes are decoded and multiplied, summed into a resident sum of products, combined with the sliding FP32 accumulator into the 63-bit frame word, then normalised and rounded to FP32; the block scale is hoisted once in the front end and re-enters only on the result exponent.

the result register is occupied and the consumer is not ready. Each stage carries an independent valid bit and instruction-identifier sideband, so the three fused-format engines accept one instruction per cycle and the interface layer restores program order through an order queue.

E. MXFP4R: dual-scale placement

The two product terms of a MXFP4R dot product carry independent scales, so (4) factors on the primary scale $s_a s_w$ and admits the residue term by a shift of $\delta = S_a - S_{ar}$. This split is realised as the `mrx4dot/mxfinal` instruction pair: `mrx4dot` forms the two scale-free sums and stages them in a single-entry buffer, writing no register, while `mxfinal` reads the three block scales and the FP32 accumulator as plain 32-bit operands, applies the scales and δ as exponent adjustments, and rounds once to nearest-even before writing the result. Placement rests on the following property of the quantiser.

Property 1. The residue block’s shared exponent does not exceed the primary’s: $S_{ar} \leq S_a$.

Proof. Under the OCP scale rule $S = \lfloor \log_2 amax \rfloor - e_{\max}$ with clipping at max_norm , the primary block’s per-element quantisation error is bounded by $2 \cdot 2^{S_a}$ at the clip corner, so $amax_{resid} < 2^{S_a+1}$ and hence $S_a - S_{ar} \geq 2$; an all-zero block gives equality at margin zero. $\square$

Property 1 reduces anchor selection to a one-hot multiplexer: the primary product is frame-resident when nonzero and the residue product slides down by δ ; otherwise the residue product is resident. Each non-resident term right-shifts with the shift saturated at that term’s own width—13 bits for a product, 25 for the signed accumulator mantissa—beyond which further shifting is a no-op. Saturating at the term width rather than at REMAIN is necessary for exactness when the required shift is small.

Two consequences of carrying two resident terms, rather than one, follow from the bounds of §IV-C. First, the frame-width margin doubles: the resident magnitude is $|\sum_i a_i w_i| + |\sum_i ar_i w_i|$, each bounded independently at the single-scale limit and coincident (both frame-resident) exactly at $\delta = 0$ —matching MXFP4’s own 38-bit frame at no added cost. Sec-

ond, the two resident terms can cancel against *each other*, not only against the accumulator: for $\delta \leq \delta_{\max} = 11$ they combine to a coefficient far smaller than either alone, so the single-scale closure applied to that residual requires $\text{REMAIN} = 36$ —the single-scale 25 plus the residue term’s maximum alignment shift of 11. Cancellation is impossible for $\delta \geq 12$, since $|\sum_i a_i w_i| 2^\delta$ then exceeds the bound either sum can take alone. Three bypasses close the frame: the accumulator dominates, exactly as in every engine; both sums are zero; or the two sums cancel exactly. The wider REMAIN—not a wider frame—is therefore the residue format’s entire extra hardware cost relative to a single-scale engine.

F. M²XFP4: on-chip top-1 re-derivation

The M²XFP4 accelerator of [3] performs top-1 decode in a unit separate from its systolic array, forwarding a value–index pair and selecting the matching weight through an 8:1 multiplexer with an auxiliary MAC for the correction term. In a coprocessor that decodes all 16 lanes in parallel, no selection is required: the top-1 index is re-derived within the datapath by a priority encoder over the subgroup magnitudes, which—since E2M1 magnitude codes are monotone in value—reduces to a 3-bit unsigned comparison. The winning lane carries a 7-bit code through a 7×5 product array while the remaining lanes use 5×5 arrays, at a cost of one widened lane per subgroup and without the separate multiplexer or correction MAC.²

V. EVALUATION

A. Setup

Each engine is synthesised out of context with Yosys [8] through the OpenROAD flow [9] at the Nangate45 45 nm node, under a common timing constraint. Functional simulation uses Verilator [10].

B. Synthesis and throughput analysis

Physical synthesis results confirm our core hypothesis: specialising hardware datapaths to narrow microscaling formats yields significant area, frequency, and throughput density advantages over wider uniform representations.

First, our measurements demonstrate that narrow fused datapaths achieve superior physical efficiency. MXFP4 emerges as the most compact and agile fused engine, requiring only $13,442 \mu\text{m}^2$ and operating at 414.9 MHz. This efficiency stems directly from its narrow four-bit multipliers and lightweight accumulation frame, compared to the 95-bit frame required by MXFP8. While M²XFP4 offers metadata refinements, its extra logic increases area and power consumption to 441.0 mW while lowering maximum frequency to 359.5 MHz. MXFP8 requires the largest area ($23,881 \mu\text{m}^2$) and lowest frequency (288.3 MHz), confirming that wide uniform formats incur

²The reference [3] is internally inconsistent on the tie-break when a subgroup has more than one maximum-magnitude element (an estimated 12% of subgroups under uniform-random data): its Algorithm 1 selects on magnitude with ties to the lowest index, while its Fig. 10 ranks through a signed value LUT that orders +6 above −6, so the two never tie and the sign decides. This design follows Algorithm 1; readers reconciling against [3]’s reported accuracy ([3]’s Table I) should note the choice.

TABLE V
PER-ENGINE OUT-OF-CONTEXT SYNTHESIS, NANGATE45 45 nm, ALL
CONSTRAINED AT A COMMON PERIOD. ZERO HOLD AND ZERO ELECTRICAL
VIOLATIONS IN EVERY RUN.

Engine	Elem./ instr.	Area (μm^2)	F_{max} (MHz)	Power (mW)	Throughput (GFLOP/s)	Density (GFLOP/(s mm ²))
MXFP4	16	13,442	414.9	246.0	13.28	988
M ² XFP4	16	16,419	359.5	441.0	11.50	701
MXFP8	8	23,881	288.3	193.0	4.61	193
m _{xr} 4dot	16	13,070	915.5	173.0	29.30	2,241
m _x final	16	11,149	441.0	148.0	14.11	1,266

m_{xr}4dot and m_xfinal are the two pipelined stages of the MXFP4R instruction pair (§IV-B). m_{xr}4dot performs scale-free reduction (29.30 GFLOP/s, 173.0 mW), while m_xfinal completes FP32 accumulation and rounding (14.11 GFLOP/s, 148.0 mW), bounding the pair's throughput at 14.11 GFLOP/s.

severe area and timing penalties that offset their element-level parallel gains.

Second, we establish that multi-instruction residue execution can overcome register file port constraints without sacrificing compute throughput. By decomposing the MXFP4R dot product into a decoupled instruction pair (m_{xr}4dot and m_xfinal), we eliminate the three-read-port register file bottleneck while unrolling combinational delay paths. The reduction stage (m_{xr}4dot) achieves 915.5 MHz, and the accumulator stage (m_xfinal) reaches 441.0 MHz. Sustaining one completed pair every two cycles yields a useful throughput of 14.11 GFLOP/s, representing a $3.06\times$ speedup over MXFP8 (Fig. V) and outperforming all single-instruction candidates.

C. Unified evaluation framework

Because prior studies evaluated microscaling formats across different process nodes, synthesis flows, and host processors, published figures cannot be directly compared to isolate format-level trade-offs. To establish a rigorous benchmark, we reimplemented all format engines within a unified framework using the same RISC-V host core (CV32E40X), identical timing constraints, the open-source Yosys and OpenROAD toolchain, and the Nangate45 45 nm PDK.

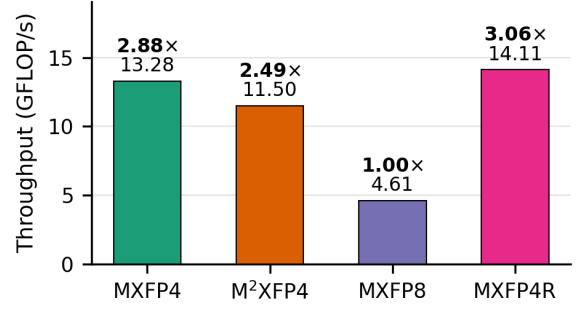
VI. CONCLUSION

This work evaluated MXFP4R, a residue MX format, by implementing it alongside MXFP4, MXFP8, and M²XFP4 as format-specialised engines within a single CV-X-IF-compliant coprocessor and characterising all four on a common core, flow, and technology node. MXFP4R does not improve on MXFP8 in compute throughput, as its dual-scale dot product requires an instruction pair; its benefit is a weight footprint equal to MXFP4 at accuracy approaching MXFP8, which favours memory-bound inference. The accompanying per-format scale-free frame keeps each engine exact to round-to-nearest-even across the full E8M0 scale range at a width set by its element format, and the coprocessor is portable across CV-X-IF-compliant hosts.

ACKNOWLEDGMENT

Use of AI. The authors used AI tools to assist with (i) drafting portions of the manuscript text, (ii) generating figures,

Useful Throughput vs. MXFP8



and (iii) writing RTL implementation code, all under the authors' direction. All AI-assisted output was reviewed, verified, and edited by the authors, who take full responsibility for the correctness and originality of the paper's content and results.

REFERENCES

- [1] Open Compute Project, "OCP microscaling formats (MX) specification, version 1.0," Open Compute Project Foundation, Tech. Rep., 2023.
- [2] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung, "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.
- [3] W. Hu, Z. Zhang, H. Zhang, C. Zhang, C. Guo, Y. Feng, T. Hu, G. Li, G. Hu, J. Wang, and J. Leng, "M²XFP: A metadata-augmented microscaling data format for efficient low-bit quantization," in *Proc. ACM Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2026, arXiv:2601.19213.
- [4] G. Islamoglu, L. Bertaccini, A. S. Prasad, F. Conti, A. Garofalo, and L. Benini, "MXDOTP: A RISC-V ISA extension for enabling microscaling (MX) floating-point dot products," in *Proc. IEEE 36th Int'l Conf. on Application-specific Systems, Architectures and Processors (ASAP)*, 2025, arXiv:2505.13159.
- [5] M. Wipfli, G. Islamoglu, N. Kunhi Purayil, A. Garofalo, and L. Benini, "VMXDOTP: A RISC-V vector ISA extension for efficient microscaling (MX) format acceleration," in *Proc. Design, Automation and Test in Europe Conf. (DATE)*, 2026, arXiv:2603.04979.
- [6] D. R. Lutz, A. Saini, M. Kroes, T. Elmer, and H. Valsaraju, "Fused FP8 4-way dot product with scaling and FP32 accumulation," in *Proc. IEEE 31st Symposium on Computer Arithmetic (ARITH)*, 2024, pp. 40–47.
- [7] OpenHW Group, "CORE-V extension interface (CV-X-IF) specification," <https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/>, 2024.
- [8] C. Wolf, "Yosys open SYnthesis Suite," <https://yosyshq.net/yosys/>.
- [9] T. Ajayi, D. Blaauw *et al.*, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Proc. Government Microcircuit Applications and Critical Technology Conference (GOMACTech)*, 2019.
- [10] W. Snyder, "Verilator: Open-source SystemVerilog simulator," <https://verilator.org>.